Does a greedy approach always produce the optimal solution?

Answer: No, a greedy approach does **not** always produce the optimal solution.

Explanation:

A greedy algorithm works by making the **locally optimal choice** at each step (the best choice right now) with the hope that these choices will lead to a **globally optimal solution**.

While this method is fast and efficient, it fails in many cases because it does not consider the future consequences of a current decision.

When does it work?

A greedy approach only guarantees the best result if the problem satisfies two specific properties:

1. **Greedy Choice Property:** A global optimum can be reached by making a local greedy choice.
2. **Optimal Substructure:** The optimal solution to the large problem contains optimal solutions to its sub-problems.

Example of Failure (0/1 Knapsack Problem):

In the **0/1 Knapsack Problem**, a greedy approach based on the highest value-to-weight ratio often fails.

- **Reason:** Because we cannot break items (we must take them as a whole), taking the "greediest" item might leave a small amount of empty space in the bag that cannot be filled by any other item. A non-greedy choice might have allowed us to combine multiple items to get a higher total value.

সহজ ব্যাখ্যা (Explanation):

Greedy Algorithm-এর কাজ করার ধরন হলো অনেকটা "**লোভী মানুষের**" মতো। সে শুধু দেখে **বর্তমানে** তার জন্য সবচেয়ে লাভজনক অপশন কোনটি। সে ভবিষ্যতের কথা চিন্তা করে না।

এই পদ্ধতিটি তখনই কাজ করে যখন নিচের দুটি বৈশিষ্ট্য থাকে:

1. **Greedy Choice Property:** বর্তমানে যেটা সেরা মনে হচ্ছে, সেটাই শেষ পর্যন্ত সেরার দিকে নিয়ে যাবে।
2. **Optimal Substructure:** বড় সমস্যার সমাধানটি ছোট ছোট সমস্যার সেরা সমাধানের মাধ্যমেই পাওয়া যাবে।

কেন এটি সব সময় কাজ করে না? (একটি বাস্তব উদাহরণ)

ধরো, তুমি একটি পাহাড়ের চূড়ায় উঠতে চাও। Greedy পদ্ধতি তোমাকে বলবে সবসময় **খাড়া উপরের দিকে** উঠতে।

- **Greedy-র ভুল:** অনেক সময় সামনে একটু নিচে নেমে তারপর বড় পাহাড়ে উঠতে হয়। কিন্তু Greedy যেহেতু নিচে নামতে জানে না (সে শুধু বর্তমান লাভ দেখে), তাই সে হয়তো একটা ছোট টিলার ওপর উঠে আটকে যাবে। সে আসল উঁচু পাহাড়ে আর পৌঁছাতে পারবে না।

পরীক্ষায় দেওয়ার মতো উদাহরণ:

- **0/1 Knapsack Problem:** এটি Greedy দিয়ে করলে ভুল উত্তর আসতে পারে (আগের প্রশ্নে আমরা যেমনটা দেখেছি)। এর জন্য **Dynamic Programming** দরকার।
- **Shortest Path:** কিছু ক্ষেত্রে Greedy ভুল পথ দেখাতে পারে যদি রাস্তার সব তথ্য আগে থেকে না থাকে।

Feature	**Merge Sort**	**Quick Sort**
Strategy	Follows the **Divide and Conquer** approach by splitting the array into two equal halves.	Follows **Divide and Conquer** by picking a **Pivot** element and partitioning the array around it.
Worst Case Complexity	Guaranteed **$O(n \log n)$**. It is very consistent.	Can be **$O(n^2)$** if the pivot selection is poor (e.g., already sorted data).
Space Complexity	**$O(n)$** (High). It requires extra temporary memory to merge subarrays.	**$O(\log n)$** (Low). It is an **In-place** algorithm and doesn't need extra arrays.
Stability	It is **Stable** (preserves the relative order of equal elements).	It is generally **Unstable**.
Preferred Data Type	Highly efficient for **Linked Lists** and very large datasets that don't fit in RAM.	Highly efficient for **Arrays** and is typically faster in practice for internal sorting.

Feature	Merge Sort	Quick Sort
Sorting Method	Most of the work happens during the **Merging** phase.	Most of the work happens during the **Partitioning** phase.

Definition: Stable Sorting Algorithm

A sorting algorithm is defined as **stable** if it preserves the **relative order** of records with equal keys (values). In other words, if two elements A and B have the same value and A appeared before B in the original unsorted list, A will definitely appear before B in the sorted list.

Example of Stability

Consider a list of students with their names and marks, where we want to sort them by **Marks**:

- **Input:** `[(Abir, 85), (Bina, 90), (Siam, 85)]`

Note that both **Abir** and **Siam** have **85** marks, and Abir is currently ahead of Siam.

1. **Stable Sort Result:** `[(Abir, 85), (Siam, 85), (Bina, 90)]`

   (Abir remains before Siam because the algorithm is stable.)

2. **Unstable Sort Result:** `[(Siam, 85), (Abir, 85), (Bina, 90)]`

   (The order of Abir and Siam is flipped, which can happen in an unstable sort.)

Algorithm	Status	Primary Reason
Bubble	**Stable**	Swaps only strictly larger adjacent values.
Insertion	**Stable**	Places equal items after existing ones during shift.
Merge	**Stable**	Prefers the "Left" element during the merge of equal values.
Selection	**Unstable**	Long-distance swaps move elements past equal ones.

Algorithm	Status	Primary Reason
Quick	**Unstable**	Swapping during partitioning ignores relative positions.
Heap	**Unstable**	The tree structure rearrangement destroys sequence.

For activity/interval scheduling problem in greedy approach, why is the earliest finish time approach chosen as the best strategy?

Why Earliest Finish Time (EFT) is the Best Strategy

In the Activity Selection Problem, the goal is to select the maximum number of non-overlapping activities. The **Earliest Finish Time** approach is the optimal greedy strategy due to the following reasons:

1. Maximizing Resource Availability

The primary objective is to leave as much time as possible for subsequent activities. By choosing the activity that finishes the earliest, we free up the resource (like a meeting room or CPU) as soon as possible. This creates the maximum amount of remaining time to accommodate more tasks.

2. The Greedy Choice Property

The EFT strategy ensures that we are always making a "safe" choice. If we have an activity A that finishes earlier than any other activity, picking A will never result in a worse solution than picking any other activity.

- Even if an optimal solution exists that doesn't include A, we can replace the first activity of that solution with A without creating any overlaps, because A finishes even earlier.

3. Failure of Alternative Strategies

Other greedy strategies often lead to sub-optimal results:

- **Earliest Start Time:** A task might start very early but have a very long duration, blocking many other potential activities.
- **Shortest Duration:** A very short activity might be positioned in such a way that it overlaps with two other potentially longer activities, reducing the total count.

Illustrative Example (Counter-Proof)

Consider three activities:

1. **Activity X:** Start 1, Finish 10
2. **Activity Y:** Start 9, Finish 11
3. **Activity Z:** Start 10, Finish 20

- **Using Shortest Duration:** We would pick **Activity Y** (duration 2). But this blocks both X and Z. Total = **1 activity**.
- **Using Earliest Finish Time:** We pick **Activity X** (finishes at 10), then we can still pick **Activity Z** (starts at 10). Total = **2 activities**.

Conclusion

Therefore, the **Earliest Finish Time** is the best strategy because it is the most "conservative" choice—it minimizes the usage of the resource at each step, directly leading to the global maximum number of activities.

নিচে exam-এ লেখার মতো করে সুন্দরভাবে সাজিয়ে দিচ্ছি 👇

Prefix Codes (Prefix-Free Codes)

Definition:
A **Prefix Code** is a type of code in which **no codeword is a prefix of any other codeword**. That is, for any two distinct codewords, one cannot be the starting part of another.

Why it is called Prefix-Free Code

A better name is **prefix-free code** because the set of codewords is completely **free from prefix conflicts**, meaning no ambiguity occurs during decoding.

Example

Not a Prefix Code:

```
A = 0
B = 01
C = 10
```

Here, `0` is a prefix of `01` → Not allowed ✗

Valid Prefix Code:

```
A = 0
B = 10
C = 11
```

No codeword is the prefix of another → Valid ✓

Advantages of Prefix Codes

1. **Unambiguous Decoding:**
   Messages can be decoded uniquely without confusion.
2. **Instant (Online) Decoding:**
   No need to look ahead; decoding can be done as bits are read.
3. **Efficient Data Compression:**
   Prefix codes are used in optimal compression techniques like
   Huffman Coding, where frequently used symbols are assigned shorter codes.

Does a greedy approach always produce the optimal solution?

No, a greedy approach does **not always** produce the optimal solution.

The **Greedy Algorithm** makes the best local choice at each step, but this does not guarantee a globally optimal solution for all problems.

It works correctly only for problems that satisfy the greedy-choice property and optimal substructure.

Example: Greedy approach not always optimal

❌ Counter Example: Coin Change Problem

Suppose coin denominations are:
`{1, 3, 4}` and we need to make **6**

◆ Greedy Approach (always pick largest coin first)

- Take 4 → remaining = 2
- Take 1 → remaining = 1
- Take 1 → remaining = 0

☞ Coins used = **4 + 1 + 1 = 3 coins**

◆ Optimal Solution

- Take 3 + 3 = 6

👉 **Coins used = 2 coins**

What stretegy select the next item(and the amount it)?

Answer (Exam style):

The strategy that selects the next item based on the best immediate (local) choice is called the **Greedy Strategy**.

In this approach, the algorithm:

- Chooses the **next item that looks best at the moment**
- Decides the amount based on maximizing profit or minimizing cost at each step
- Never reconsiders previous choices

Why 0-1 Knapsack is Solved Using Dynamic Programming

The 0-1 Knapsack problem is solved using Dynamic Programming (DP) because it satisfies two main properties that make DP more effective than Greedy or Brute Force:

1. Optimal Substructure

The problem can be broken down into smaller, simpler subproblems. To find the optimal solution for a total capacity of **$W=7$**, we first calculate the optimal solutions for all smaller capacities (**$w = 1, 2, 3... 6$**). The global maximum value is built using the optimal results of these smaller capacities.

2. Overlapping Subproblems

During the calculation, the same subproblems are solved multiple times. For example, to decide whether to include Item 4, we need the results already calculated for Items 1, 2, and 3 at various weight levels. Instead of recalculating these values, DP stores them in a **Matrix (Table)** to save time and computation.

3. Failure of the Greedy Approach

The Greedy approach (picking items based on highest value or best value-to-weight ratio) fails in 0-1 Knapsack because we cannot take "fractions" of an item.

- A Greedy choice might pick a high-value item that takes up too much space, preventing us from picking two slightly smaller items that together would have provided more total value.

- DP explores all possible combinations through the matrix to ensure the absolute **Optimal Price** is found.

Do we consider actual running time for comparing algorithms? Justify your answer

No, we do not consider the actual running time for comparing algorithms.

The actual running time of an algorithm depends on many external factors such as the processor speed, memory capacity, compiler efficiency, programming language, and operating system. Because of these factors, the same algorithm may take different amounts of time on different machines, even for the same input size. Therefore, actual running time is not a reliable or fair method for comparing algorithms.

Instead, we analyze algorithms using **asymptotic notation (Big-O notation)**, which focuses on how the running time grows with respect to the input size. This method is independent of hardware and implementation details, making it more consistent and suitable for comparison.

For example, an algorithm with time complexity O(n2) will always grow faster than an algorithm with O(n) for large input sizes, regardless of the machine used.

Do we consider actual running time for comparing algorithms? Justify your answer

Answer:

The given description refers to the **Divide and Conquer algorithm design technique**.

In this approach, a problem is solved in three main steps:

1. Divide:
The original problem is divided into smaller sub-problems that are similar to the original problem but smaller in size.

2. Conquer:
Each sub-problem is solved recursively. If a sub-problem becomes small enough (called the **base case**), it is solved directly without further division. The recursion continues until all sub-problems reach the base case.

3. Combine:
The solutions of all sub-problems are combined together to form the solution of the original problem.

The **base case** is the simplest form of the problem that can be solved directly, and it stops the recursion process.

Conclusion:
Divide and Conquer works by breaking a large problem into smaller ones, solving them recursively until reaching the base case, and then combining the results to obtain the final solution.

Define Complete Binary tree with example.
A **Complete Binary Tree** is a binary tree in which:

- All levels are completely filled **except possibly the last level**, and
- The last level has all its nodes as **far left as possible** (no gaps between nodes).

Feature	Linear Data Structure	Non-Linear Data Structure
Arrangement	Elements are arranged in a sequential (one after another) order	Elements are arranged in a hierarchical or connected structure
Traversal	Traversed in a single level (one path)	Traversed in multiple levels (multiple paths)
Relationship	Each element has at most one predecessor and one successor	A node can have multiple children/links
Complexity	Simpler to implement and understand	More complex structure
Memory usage	Usually uses sequential memory	May use non-sequential memory

✓ Linear Data Structure:

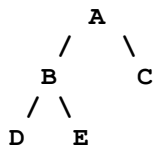
- Array
- Linked List
- Stack
- Queue

☞ Example: `A → B → C → D`

✓ Non-Linear Data Structure:

- Tree
- Graph

☞ Example (Tree):



◆ Polish Notation (Prefix Notation):

Polish notation is a method of writing arithmetic expressions in which the **operator is written before the operands**. In this notation, parentheses are not required because the order of operations is unambiguous.

Example:

- Infix: A + B
- Prefix: + A B
- Infix: (A + B) * C
- Prefix: * + A B C

◆ Reverse Polish Notation (Postfix Notation):

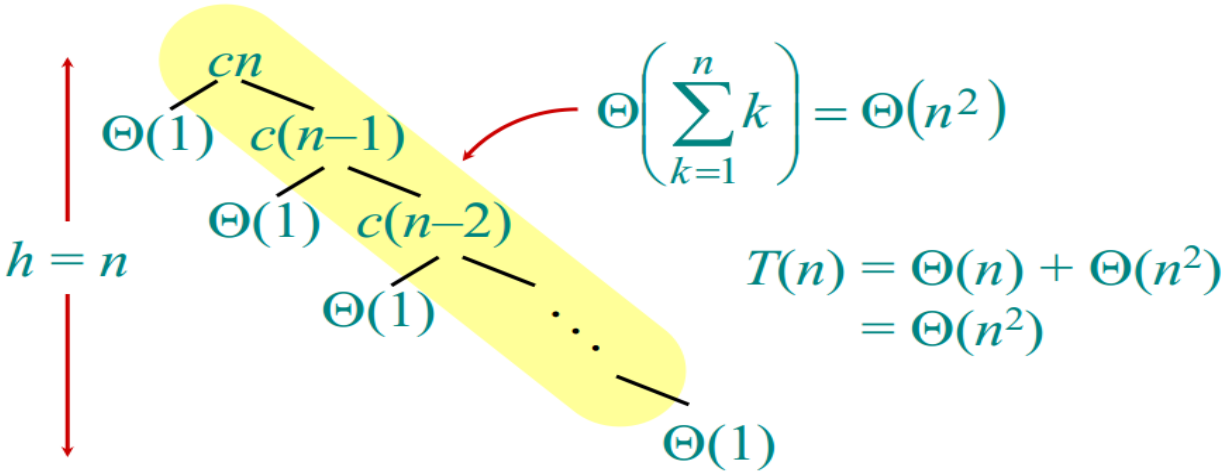
Reverse Polish notation is a method of writing arithmetic expressions in which the **operator is written after the operands**. Like prefix notation, it also does not require parentheses.

Example:

- Infix: A + B
- Postfix: A B +
- Infix: (A + B) * C
- Postfix: A B + C *

Quick Sort Worst Case complexity:

$$T(n) = T(1) + T(n-1) + cn$$



১. মূল সমীকরণ (Recurrence Relation)

এখানে দেওয়া আছে: $T(n) = T(1) + T(n-1) + cn$

এর মানে হলো:

- cn: বর্তমান ধাপে কাজ করার খরচ (পার্টিশন করা)।
- $T(1)$: একটা এলিমেন্ট আলাদা হয়ে গেল (বাজে পার্টিশন)।
- $T(n-1)$: বাকি $n-1$ টা এলিমেন্ট পরের ধাপের জন্য রয়ে গেল।

২. রিকার্সন ট্রি (Recursion Tree)

ছবিতে দেখো, ট্রি-টা একদিকে হেলে আছে। কারণ প্রতিবার মাত্র ১টা এলিমেন্ট কমছে।

- ১ম ধাপে কাজ: cn

- ১ম ধাপে কাজ: cn
- ২য় ধাপে কাজ: $c(n-1)$
- ৩য় ধাপে কাজ: $c(n-2)$
- এভাবে করতে করতে একদম শেষে গিয়ে কাজ হবে $c(1)$ বা কনস্ট্যান্ট।

৩. মোট কাজের যোগফল (The Summation)

এখন পুরো ট্রি-তে মোট কত কাজ হলো তা বের করতে আমাদের সবগুলোকে যোগ করতে হবে:

$$\text{Total Work} = cn + c(n-1) + c(n-2) + \cdots + 2c + c$$

এখান থেকে c কমন নিলে থাকে:

এখান থেকে c কমন নিলে থাকে:

$$c \times (n + (n-1) + (n-2) + \cdots + 1)$$

৪. কেন এটা $O(n^2)$?

আমরা জানি, 1 থেকে n পর্যন্ত সংখ্যার যোগফলের সূত্র হলো: $\frac{n(n+1)}{2}$
তাহলে আমাদের সমীকরণটি দাঁড়ায়:

$$\text{Total Work} = c \times \frac{n(n+1)}{2}$$

$$\text{Total Work} = c \times \frac{n^2 + n}{2}$$

এখন বিগ-ও (O) নোটেশনের নিয়ম অনুযায়ী:

1. আমরা কনস্ট্যান্ট (c এবং $/2$) বাদ দিয়ে দিই।
2. আমরা শুধু সবচেয়ে বড় পাওয়ার বা ঘাতটি রাখি (এখানে n^2 বড়, n ছোট)।

তাই শেষ পর্যন্ত আমরা পাই: $O(n^2)$।

Why are more frequent characters higher in the tree in Huffman coding?
"In Huffman coding, more frequent characters are placed higher in the tree to assign them **shorter binary codes**. Since these characters appear most often, shorter codes significantly reduce the **total size** of the compressed data, ensuring maximum efficiency."